Workshop 6

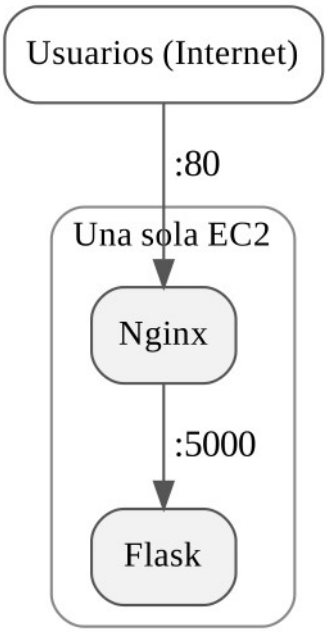
Maria Lucía Castillo García

Índice de contenido

Parte 1: Arquitectura inicial....1
Parte 2: Evolución....2
Parte 3: Stateless....2
Parte 4: Persistencia....3
Parte 5: Auto Scaling....3
Parte 6: Diseño....4
Comparación: una instancia vs múltiples instancias....5
Escalabilidad horizontal....5
Reto final....5

Parte 1: Arquitectura inicial

Arquitectura de los talleres anteriores: una sola EC2



1) ¿Qué sucede si EC2 falla?

- La aplicación se cae completa, porque Nginx y Flask están en esa única máquina. Nadie puede entrar hasta que la arregle o cree otra.

2) ¿Qué sucede si aumenta el tráfico?

- Una `t3.micro` tiene 2 vCPU y menos de 1 GB de RAM. Si llegan muchas peticiones, se pone lenta o deja de responder, porque no hay otra máquina que reparta la carga.

3) ¿Dónde existe un Single Point of Failure?

- En la EC2. Todo depende de ella: si falla, falla todo. Lo mismo pasa con la zona de disponibilidad donde está.

Parte 2: Evolución

1) ¿Qué componente distribuye el tráfico?

- El Load Balancer. Recibe todas las peticiones y las reparte entre las EC2, parecido a lo que hacía Nginx con el round robin en los talleres de Docker, pero entre máquinas distintas.

2) ¿Dónde se ejecuta cada instancia?

- Cada EC2 es una máquina aparte, y conviene que cada una esté en una zona de disponibilidad diferente de la región (por ejemplo `us-east-1a`, `us-east-1b` y `us-east-1c`).

3) ¿Por qué conviene distribuir instancias entre Availability Zones?

- Porque cada zona es un centro de datos separado. Si una zona tiene un problema, las instancias de las otras siguen funcionando y el Load Balancer les manda el tráfico a ellas.

Parte 3: Stateless

Si la sesión de un usuario está guardada solo en la memoria de EC2-1 y la siguiente petición llega a EC2-2, esa instancia no sabe quién es el usuario. Por ejemplo, le pediría que vuelva a iniciar sesión o se perdería lo que tenía en el carrito.

Por eso las aplicaciones deben ser stateless: ninguna instancia debe guardar datos propios del usuario. La sesión se guarda en un lugar compartido (una base de datos o algo como Redis), y así cualquier EC2 puede atender cualquier petición.

Parte 4: Persistencia

No, no son el mismo archivo. Cada EC2 tiene su propio disco, así que `archivo.txt` en EC2-1 y en EC2-2 son dos archivos distintos que pueden tener contenido diferente. Además, si una instancia se borra (por ejemplo cuando el Auto Scaling la quita), su archivo se pierde.

Propuesta: que las instancias no guarden nada importante en su disco.

- Datos estructurados (usuarios, registros): en una base de datos administrada, como Amazon RDS.
- Archivos (fotos, PDFs): en Amazon S3, que todas las instancias pueden leer y escribir.
- Si de verdad se necesita una carpeta compartida: Amazon EFS, que se monta igual en todas las EC2.

Parte 5: Auto Scaling

1) ¿Qué problema resuelve Auto Scaling?

- Ajusta la cantidad de EC2 según la demanda: agrega instancias cuando hay mucho tráfico y las quita cuando baja, para no pagar de más. También reemplaza sola una instancia que falle. Para crear las nuevas usa una plantilla (*launch template*), que puede llevar un User Data como el del Workshop 5.

2) ¿Qué diferencia existe entre escalar verticalmente y horizontalmente?

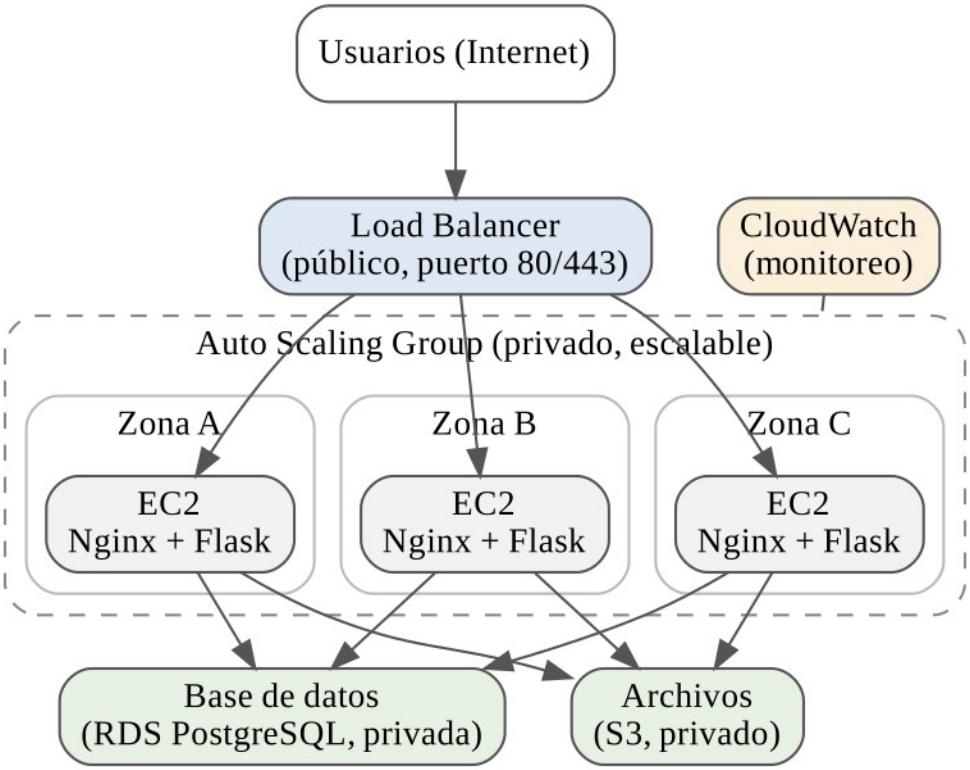
- Vertical es hacer más grande la misma máquina, por ejemplo pasar de `t3.micro` a una instancia con más CPU y RAM. Tiene un límite y normalmente hay que apagarla para cambiarla.
Horizontal es agregar más máquinas iguales y repartir el trabajo entre ellas. No tiene ese límite y, si una falla, las demás siguen.

3) ¿Qué métrica podría utilizarse como señal de escalamiento?

- El uso de CPU (por ejemplo, agregar una instancia si el promedio pasa del 70 %) o la cantidad de peticiones por instancia que recibe el Load Balancer. Estas métricas se ven en CloudWatch.

Parte 6: Diseño

Diagrama de la arquitectura propuesta



Tipo	Elementos
Públicos	Solo el Load Balancer, por el 80/443.
Privados	Las EC2 (solo reciben tráfico del Load Balancer), la base de datos y el bucket de S3.
Escalables	Las EC2 con Flask, dentro del Auto Scaling Group.
Persistentes	La base de datos (RDS) y los archivos (S3). Las EC2 se pueden borrar sin perder nada.
Monitoreados	Todo, con CloudWatch: CPU de las EC2, peticiones y errores del Load Balancer, y la base de datos.

Comparación: una instancia vs múltiples instancias

Aspecto	Una instancia	Múltiples instancias
Si falla	Se cae todo.	Las demás siguen atendiendo.
Mucho tráfico	Se pone lenta.	Se reparte entre varias y se pueden agregar más.
Datos	Pueden quedar en el disco local.	Tienen que estar en un lugar compartido (RDS, S3).
Complejidad	Muy simple de montar.	Necesita Load Balancer, Auto Scaling y app stateless.
Costo	Barato.	Más caro, pero se ajusta a la demanda.

Escalabilidad horizontal

Escalar horizontalmente es atender más usuarios agregando más servidores iguales, en vez de comprar uno más grande. El Load Balancer reparte las peticiones y el Auto Scaling decide cuántos servidores hacen falta. Para que funcione, la aplicación tiene que ser stateless y guardar los datos fuera de las EC2.

Reto final

Así fue evolucionando el laboratorio y lo que cambia en cada paso:

1. **Docker local:** la aplicación corría en mi computador con Docker Compose (talleres de `docker_03`).
2. **Docker en EC2:** los mismos contenedores, pero en un servidor de AWS accesible desde Internet (Workshop 3).
3. **Múltiples EC2:** varias instancias iguales, creadas con un User Data para que salgan idénticas (Workshop 5).
4. **Load Balancer:** un punto de entrada que reparte el tráfico entre las instancias.
5. **Auto Scaling:** la cantidad de instancias se ajusta sola según la demanda.

Esto se relaciona con Journey to Cloud (J2C), que es el camino para llevar una aplicación a la nube por etapas en lugar de hacerlo todo de una vez. Primero se mueve la aplicación casi igual a como estaba (pasar de Docker local a Docker en EC2). Después se ajusta para aprovechar la nube (varias instancias con Load Balancer). Y al final se moderniza para usar servicios administrados que escalan solos.

Propuesta de evolución hacia una arquitectura más administrada

El siguiente paso sería que AWS administre más partes, para no tener que cuidar servidores a mano:

- Guardar las imágenes de Docker en Amazon ECR, como en el ejercicio de la clase 05.
- Correr los contenedores en Amazon ECS con Fargate, en vez de instalar Docker en cada EC2.
- Usar Amazon RDS para PostgreSQL y S3 para los archivos.
- Usar un Application Load Balancer con HTTPS y vigilar todo con CloudWatch.